

Git Guild SOLID 检查清单

1. 检查对象

检查对象为 docs/P3/核心类图 中的 AI 原始类图，以及基于 P1 需求文档、P2 架构设计文档修订后的 docs/P3/核心类图.md 和 docs/P3/类成员说明.md。

AI 原始类图覆盖了用户、仓库、Issue、PR、任务、提交、审核、通知和推荐等主干对象，但存在实体职责过重、外部平台写死、核心模块缺失、接口抽象不足等问题。

2. SOLID 总览表

原则	是否违规	违规说明	修正方案
S - 单一职责	是	实体类混入认证、同步和发送	拆出认证、适配器和通知发送服务
O - 开闭原则	是	平台和推荐规则被写死	引入适配器接口和推荐策略接口
L - 里氏替换	存在潜在风险	平台差异不应通过实体继承表达	用接口实现替代平台实体继承
I - 接口隔离	是	缺少小接口，易形成大服务	按外部平台、推荐、通知拆分接口
D - 依赖倒转	是	高层业务容易依赖具体平台 API	业务模块依赖抽象接口和领域对象

3. 详细违规记录

编号	原则	原始设计问题	修正设计
1	S	<code>User</code> 包含登录、注册、绑定 Gitea 行为	拆出 <code>AuthService</code> 和 <code>CodeHostAccountBinding</code>
2	S	<code>Repository</code> 包含导入和同步行为	由 <code>CodeHostAdapter</code> 负责外部同步
3	S	<code>Notification</code> 包含 <code>send()</code>	由 <code>NotificationService</code> 和发送器负责投递
4	S	成果审核和管理员审核未区分	增加 <code>AdminReviewRecord</code>
5	O	<code>Repository</code> 写死 GitHub 字段和方法	使用 <code>sourceUrl</code> 、 <code>hostType</code> 和适配器接口
6	O	推荐规则固定在 <code>RecommendationService</code> 中	使用 <code>RecommendationStrategy</code> 扩展规则
7	O	通知通道固定为单个发送方法	使用 <code>NotificationSender</code> 扩展通道
8	L	平台差异可能被设计成仓库子类	不用实体继承，统一走平台适配器
9	I	原图缺少接口边界	拆分平台、推荐、通知三个小接口
10	D	任务和仓库类容易直接依赖 GitHub/Gitea API	高层模块只依赖 <code>CodeHostAdapter</code> 抽象
11	S / O	<code>Quest</code> 缺少分类、标签和筛选条件	增加 <code>QuestCategory</code> 、 <code>QuestTag</code> 、筛选条件对象
12	S	<code>User -- Quest</code> 无法表达接取历史	增加 <code>QuestAssignment</code>
13	S	<code>ReviewRecord.feedback</code> 只能保存一段文字	增加 <code>ReviewItem</code> 表达逐项意见
14	D	推荐服务没有稳定的推荐结果对象	增加 <code>RecommendationResult</code>
15	S	成长激励模块缺失	增加成长档案、XP 流水、贡献记录和成长服务

4. 修正后设计对 SOLID 的满足情况

4.1 S - 单一职责

修正后设计将实体、服务和外部适配职责拆开：

`User` 只保存用户身份、角色和权限判断。

`AuthService` 处理注册、登录、改密。

`Repository` 只保存仓库元数据和同步状态。

`CodeHostAdapter` 处理外部平台导入、同步和 Webhook。

`Notification` 只保存通知数据。

`NotificationService` 与 `NotificationSender` 处理通知投递。

结论：主要类的职责边界清晰，满足单一职责原则。

4.2 O - 开闭原则

修正后设计在两个变化点上保留扩展能力：

新增代码托管平台时，实现新的 `CodeHostAdapter`。

新增推荐规则时，实现新的 `RecommendationStrategy`。

新增通知渠道时，实现新的 `NotificationSender`。

结论：对外部平台、推荐规则、通知渠道这三类高变化点基本满足开闭原则。

4.3 L - 里氏替换

修正后没有使用实体继承表达平台差异，而是使用接口实现：

`GitHubImportAdapter` 与 `GiteaAdapter` 都可以替换为 `CodeHostAdapter` 使用。

`TechStackMatchStrategy` 与 `GrowthStageStrategy` 都可以替换为 `RecommendationStrategy` 使用。

`InAppNotificationSender` 与 `EmailNotificationSender` 都可以替换为 `NotificationSender` 使用。

结论：替换关系建立在稳定接口上，降低了违反里氏替换原则的风险。

4.4 I - 接口隔离

修正后没有设计一个覆盖所有能力的大接口，而是按职责拆小：

`CodeHostAdapter` 只负责外部代码托管能力。

`RecommendationStrategy` 只负责推荐评分和推荐理由。

`NotificationSender` 只负责通知发送和类型支持判断。

结论：接口粒度较小，调用方无需依赖不必要的方法，满足接口隔离原则。

4.5 D - 依赖倒转

修正后高层业务模块依赖抽象而非具体实现：

任务和审核模块依赖 `CodeHostAdapter`，不直接依赖 GitHub/Gitea SDK。

推荐服务依赖 `RecommendationStrategy`，不直接绑定某一种推荐算法。

通知服务依赖 `NotificationSender`，不直接绑定邮件或站内通知实现。

结论：核心业务模块与外部平台、推荐算法、通知通道解耦，满足依赖倒转原则。

5. 违规数量统计

原则	发现问题数	主要问题类型
S - 单一职责	7	实体类承担服务行为，审核与通知职责混杂
O - 开闭原则	3	外部平台、推荐规则、通知渠道缺少扩展点
L - 里氏替换	1	存在用继承表达平台差异的潜在风险
I - 接口隔离	1	缺少小接口，容易形成大服务
D - 依赖倒转	3	高层业务容易直接依赖外部平台实现
合计	15	需要通过服务拆分、接口抽象和设计模式修正

6. 最终结论

AI 原始类图适合作为领域对象发现的初稿，但不能直接进入详细设计。它最大的问题是把实体类、服务行为和外部平台实现混在一起，导致后续实现会快速产生高耦合。

修正后的类图通过适配器模式、策略模式和发送器接口拆分变化点，使核心业务保持在模块化单体内部，同时保留后续扩展 Gitea/GitHub 集成、推荐规则和通知渠道的空间。